FYSN33

Project 1

Author
Axel Cerne

February 16, 2026

1 Program structure

The project is organized as:

```
.
├── main.py
├── sod_shock.py
├── planet_collision.py
├── 3D_testing.py
└── results/
    ├── sod_all_fields.mp4
    ├── ...
    └── collision3.mp4
```

The computational core is implemented in `main.py`, while the other scripts contain problem-specific initialization, solver setup, plotting and animations.

1.1 Core design (`main.py`)

The computational core is structured around three main classes:

1.1.1 `class SPHsystem`

This class represents the particle system. The full state of the system is represented by a "state vector", which in this case is a $(N, 2d+2)$ matrix:

$$S \in \mathbb{R}^{N\times(2d+2)},$$

where N is the particle number and $d = 1, 2, 3$ is the dimension, so that each row contains the coordinates of the i:th particle $(\boldsymbol{r}_i, \boldsymbol{v}_i, \rho_i, e_i)$.

The class constructor takes parameters particle number N, dimension d, a kernel object `kernel`, and a default heat capacity constant $\gamma = 1.4$. It also preallocates the state matrix S and a vector of particle masses $m \in \mathbb{R}^N$.

The class provides

- Getters for position $\boldsymbol{r}_i$, velocity $\boldsymbol{v}_i$, density ρ_i, and internal energy e_i.
- A "geometry" method `geom()` which computes pairwise position and distance between particles i and j.
- Density summation
$$\rho_i = \sum_j m_j W(|\boldsymbol{r}_i - \boldsymbol{r}_j|, h).$$
- Equations of state for pressure and sound speed:
$$P = (\gamma - 1)\rho e$$
$$c = \sqrt{(\gamma - 1)e}.$$

The class thus contains all the particle data and thermodynamic relations.

1.1.2 `class cubicSplineKernel`

This class implements a specific SPH smoothing kernel: the cubic spline kernel, W, and its gradient, ∇W. It depends only on

- Spatial coordinates $\boldsymbol{r}_{ij}$ and $|\boldsymbol{r}_{ij}|$.
- Smoothing length, h.

The kernel is attached to `SPHsystem` at initialization, so that W and ∇W are given by

```
system.kernel.W()
system.kernel.gradW()
```

The two classes are used together extensively, for example:

```
...
    def momentum_equation(self,system):
        rij, rij_norm = system.geom()

        # collect kernel gradient from system
        gradW = system.kernel.gradW(rij,rij_norm)
        ...
```

1.1.3 `class NSequations`

This class implements the Navier-Stokes equations in their SPH form:

- momentum equation
- energy equation
- artificial viscosity
- (optional) self-gravity

Given an `SPHsystem` object, the class methods `momentum_equation()` and `energy_equation()` returns

$$\frac{d\boldsymbol{v}}{dt} \in \mathbb{R}^{N\times d} \quad \text{and} \quad \frac{de}{dt} \in \mathbb{R}^{N},$$

respectively.

1.2 Time integration

Time integration of the Navier-Stokes equations is done using `scipy.integrate.solve_ivp()`. This method requires two vectors `fun` $= dy/dt$, and y_0. In return, `solve_ivp()` gives the function $y(t)$. To satisfy this condition, we define a helper function `RHS(t,S_flat)`.

The `RHS()` function takes a flattened state vector and:

1. Reconstructs the state matrix
2. Recomputes density vis `density_summation()`
3. Computes time derivatives $d\boldsymbol{r}/dt =$ `system.v`, and $d\boldsymbol{v}/dt$, de/dt via `NSequations`
4. returns the flattened vector dS/dt.

`solve_ivp()` can thus be used with $y_0 = S_0$ (initial state matrix), $dy/dt =$ `RHS`(t, S). It returns the (flattened) state matrix S for each t in a specified time-span (linspace).

1.3 Spin

The final method in `main.py` is a simple helper function which adds constant rotational velocity to particles in the state matrix. Given an `SPHsystem` object, a period T and `indices`, it:

1. Computes $\boldsymbol{\omega} = (0, 0, \omega_z)$, where $\omega_z = \frac{2\pi}{T}$.
2. Computes the center of mass radius for particles indexed by `indices` according to

$$\boldsymbol{r}_{com} = \frac{\sum_i m_i \boldsymbol{r}_i}{\sum_i m_i}.$$

3. Computes the rotational velocity $\boldsymbol{v}_{rot} = \boldsymbol{\omega} \times (\boldsymbol{r}_i - \boldsymbol{r}_{com})$
4. Adds this velocity to `system.v` for the indexed particles.

This method is used before solving the initial value problem in `collision.py`, giving the particles initial rotational velocity, see for example `planet300_diskformation.mp4` in `results/`.

2 Results and Discussion

2.1 1D Sod shock problem

The results for the 1D Sod shock problem are shown in Fig. 1. To generate this figure, an `SPHsystem` object was initialized with 400 particles, 320 of which were placed on the left hand side of the origin (including the origin), and the remaining on the right hand side of the origin, respectively (excluding the origin). The spacing between particles on the left was set to $dx_L = 0.001875$ m, and on the right $dx_R = 0.075$. The corresponding vectors containing positions x were concatenated.

The `system` object was initialized with a cubic spline kernel with a smoothing length $h = 0.01$.

The state matrix, S, was initialized with the concatenated x array, a list of equal masses for each particle, $m_i = 0.001875$ kg, and initial density was computed using the density summation method on `system`. Initial energy was set to 2.5 J/kg for the left region, and 1.795 J/kg for the right region.

For the time integration, a linspace with $N_t = 40$ time points separated by $dt = 0.005$ was used. The `solve_ivp()` method was then set up using the `RHS()` function, and the integration method used was RK45.

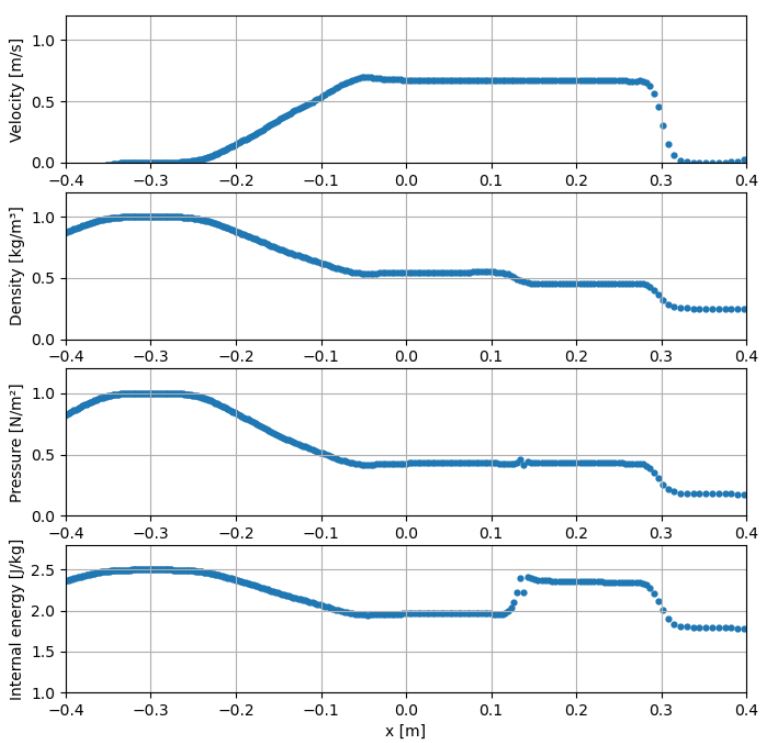

Figure 1: Sod shock solution for all field versus x.

Figure 1 shows the velocity, density, pressure, and internal energy as a function of x at the last (40:th) time-step. We clearly see a pressure shock-wave at $x \sim 0.15$ m traveling towards the less dense right region. The results also agree with the theoretical results shown in the lecture slides. Note that since we have not enforced any physical boundary conditions, we see that density, pressure and internal energy tapers off towards the left side of the respective panels, meaning the particles on the left moves away (towards the left) from the bulk, initially high density side.

I also found that the results were highly sensitive to the smoothing length h, which was first calculated using the given formula and the lower density region. I however found that simply putting it to 0.01 gave the cleanest result.

We also see a "step-wave" in internal energy which travels towards the right along with the pressure, and density fronts (easier to see in the appended .mp4 file). This makes sense, since the internal energy change is proportional to p/ρ^2 in the energy equation. Pressure also increases with increased density and energy, as $P = (\gamma - 1)\rho e$. My guess to why the waves are flat is that the kernel filters out high changes in internal energy change. When two particles are at the same place, the felt gradient $\nabla_i W_{ij}$ goes to zero, smoothing out the particle-particle exchange of internal energy.

2.2 3D Planet collisions

The 3D planet collision problem was initialized by reading position, velocity, mass, density, and pressure data from the provided "Planet300.dat" data file. Internal energy was the calculated by inverting the pressure equation of state.

Planet collision was carried out by copying the "Planet300.dat" data set, and changing every x coordinate with a constant value. All particles in both data sets were given an initial velocity component v_x (for simplicity) with opposite signs. I also shifted the relative y positions of the two bodies, added spin to both and plotted the solutions with a heat-map for density, see Fig. 2. In general I played around with this a bit to get nice looking behavior, and a few example (successful and unsuccessful) videos can be seen under `results/`. One video with 600 particles took roughly 45 min to generate, so I did not try the larger files. The main place that could be optimized for time in my code is, I believe, the fact that I compute $\boldsymbol{r}_{ij}$ and $|\boldsymbol{r}_{ij}|$ in many places for each `RHS()` call. Preferably, only one such calculation should be done, and the result fed into every dependent function per `RHS()` call.

Again, the results were found the be highly sensitive to h. With a smaller value $h \sim 1e6$, the inner particles clustered together very quickly, initially leaving rings of outer particles that slowly caught up. With a large $h \sim 1e8$ the density/pressure was larger than the inwards gravitational force and planets did not form at all. I settled on $h = 5e6$ after trial and error.

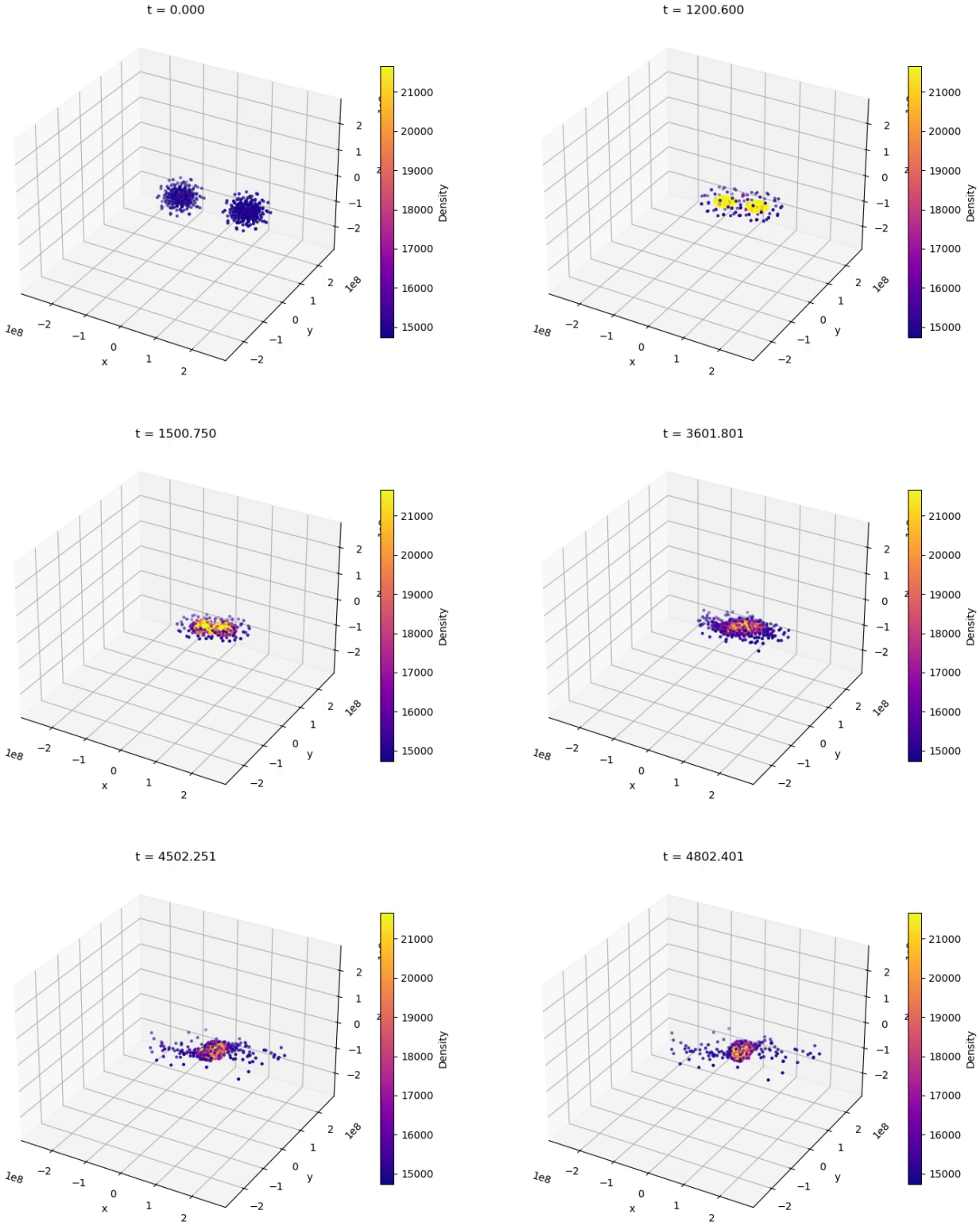

Figure 2: Snapshots of the 3D planet collision at selected times.